

Voyage Log Parser

Difficulty: Medium

Expect Time: 60 min

Author: Wayne, Hsu

A deep-sea exploration submarine produces a single-line compressed **Voyage Log** string after each mission, recording the ship name, equipment, hazards encountered at each stop, items collected, and experience gained.

Your task is to implement a parser that converts this string into a structured object and provides several query methods.

Basic Lexical Rules

- **ALPHA**: An uppercase or lowercase English letter, A–Z or a–z
- **DIGIT**: A single decimal digit, 0–9
- **ALPHANUM**: **ALPHA** or **DIGIT**
- **NAME**: A string beginning with **ALPHA**, followed by zero or more **ALPHANUM** or space characters, and not ending with a space
 - Valid examples: `Nova Shell`, `Pearl Ridge`, `Grade 3 Ore`, `Alert 9`
- **TIME**:
 - A sequence of **DIGIT** characters representing hours, e.g. `9`, `23`
 - Two **DIGIT** sequences separated by `:`, representing hours and minutes, e.g. `7:15`, `24:02`
- **DELIM** = `[ , ] , ? , @ , : , $ , & , %` — reserved separator characters; these never appear within any **NAME** token.

Log Structure

```
<SubmarineName>"["<Equipment>"]"<DepartTime><STOP><STOP>...
```

- **SubmarineName**

Composed of **NAME**.

Example:

I. `Nova Shell`

II. `Abyss Rider`

- **Equipment**

Four **DIGIT** characters enclosed in `[ ]`, each ranging from 1 to 4.

Example:

I. `[2341]`

II. `[4442]`

- **DepartTime**

TIME. The time the submarine departed from base.

- **STOP**

`["?"<Hazard>"]@<Location>": "<ArrivalTime>"$<Item>["&"<Item>"]%"<EXP>": "<LeaveTime>`

One or more STOPs follow DepartTime consecutively with no separator between them.

- Hazard **<optional>** :
NAME preceded by **?**. Zero or more hazards may appear, each introduced by its own **?**.
- Location:
NAME preceded by **@**.
- ArrivalTime:
TIME preceded by **:**.
- Item:
NAME followed by **:** and a **DIGIT** sequence. The first item is preceded by **\$**; each additional item is preceded by **&**.
- EXP:
A **DIGIT** sequence preceded by **%**.
- LeaveTime:
TIME preceded by **:**.

Example:

I. **@Pearl Ridge:8:21\$Coral:43%14352:10:55** — no hazard, one item

II. **?Fog@Ember Isle:4:11\$Frost Shard:3303&Sea Glass:3891%64785:6:36** — one hazard, two items

III. **?Coral Maze?Jellyfish Bloom@Grid 9 Basin:19:07\$Tide Moss:1550173099&Echo Stone:213035152&Grade 3 Ore:1602893089%957201546:21:21** — two hazards, three items

Methods

A `display` method is already implemented in `VoyageLog.h`. After implementing `parse`, you can call it to print the parsed log in a human-readable format and verify your results.

`parse`

Parses a compressed voyage log string into a `VoyageLog` object. A **TIME** token may be written as hours-only (e.g. `7`) or as `H:MM` (e.g. `7:15`); store it as total minutes (hours $\times$ 60 + minutes, minutes default to 0 if omitted).

`getLocationAt`

Returns the submarine's status at the given elapsed time in minutes.

- Before `departMinutes`: `"Not yet departed"`
- Travelling toward a stop: `"En route >>>"` + destination name
- During a stop's stay (`arriveMinutes` $\leq t <$ `leaveMinutes`): the location name
- At or after the last stop's `leaveMinutes`: `"Returning home"`

`getExpAt`

Returns cumulative EXP earned up to and including the given time. EXP from a stop is counted starting at its `arriveMinutes`. Use `long long` to avoid overflow.

`encode`

Re-encodes the object back to a compressed single-line string in the same format as the input, except all times are written as plain integer minutes instead of `H:MM`.

Examples

TEST_CASE("Case1"): Basic Parse — Single Stop, No Hazard, Single Item

- Sub-case 1

```
Nova Shell[2341]7:15@Pearl Ridge:8:21$Coral:43%14352:10:55
```

Field	Raw	Parsed
shipName	Nova Shell	"Nova Shell"
equipment	[2341]	{2, 3, 4, 1}
departMinutes	7:15	435
stops[0].location	Pearl Ridge	"Pearl Ridge"
stops[0].arriveMinutes	8:21	501
stops[0].hazards	—	{}
stops[0].items	Coral:43	{{"Coral", 43}}
stops[0].exp	14352	14352
stops[0].leaveMinutes	10:55	655

- Sub-case 2

```
Subber[3221]18:30@Jade Grotto:19:24$Sea Glass:31%13235:21:54
```

Field	Raw	Parsed
shipName	Subber	"Subber"
equipment	[3221]	{3, 2, 2, 1}
departMinutes	18:30	1110
stops[0].location	Jade Grotto	"Jade Grotto"
stops[0].arriveMinutes	19:24	1164
stops[0].items	Sea Glass:31	{{"Sea Glass", 31}}
stops[0].exp	13235	13235
stops[0].leaveMinutes	21:54	1314

- Sub-case 3 — DepartTime as integer hours only (no :MM)

```
Blue Phantom[2144]7@Silver Lagoon:8:53$Saltstone:51%12429:10:47
```

Field	Raw	Parsed
shipName	Blue Phantom	"Blue Phantom"
equipment	[2144]	{2, 1, 4, 4}
departMinutes	7	420
stops[0].location	Silver Lagoon	"Silver Lagoon"
stops[0].arriveMinutes	8:53	533
stops[0].items	Saltstone:51	{{"Saltstone", 51}}
stops[0].exp	12429	12429
stops[0].leaveMinutes	10:47	647

TEST_CASE("Case2"): Multi-Stop Parse with getLocationAt , getExpAt , and encode

- Sub-case 1

```
Leviathan[4111]23?Deep Current@Sunken Arch:23:42$Batch 0 Kelp:1434009068&Grade 3
```

Top-level:

shipName	equipment	stops.size()	departMinutes
"Leviathan"	{4,1,1,1}	3	1380

Stops:

#	location	arriveMinutes	hazards	items	exp	leaveMinutes
0	"Sunken Arch"	1422	Deep Current	Batch 0 Kelp:1434009068, Grade 3 Ore:1811080350	515381578	1457
1	"Crystal Cove"	1498	Thermal Vent	Pearls:1416305517, Amber:1257600446	711414429	1664
2	"Twin Peaks"	1717	Pressure Wave	Ores:1086811674, Tier 8 Fossil:1569175984	142203829	1795

getLocationAt (selected):

minutes	return value
0	"Not yet departed"
1379	"Not yet departed"

minutes	return value
1380	"En route >>>Sunken Arch"
1421	"En route >>>Sunken Arch"
1422	"Sunken Arch"
1456	"Sunken Arch"
1457	"En route >>>Crystal Cove"
1497	"En route >>>Crystal Cove"
1498	"Crystal Cove"
1663	"Crystal Cove"
1664	"En route >>>Twin Peaks"
1716	"En route >>>Twin Peaks"
1717	"Twin Peaks"
1794	"Twin Peaks"
1795	"Returning home"
1895	"Returning home"

`getExpAt` (selected):

minutes	cumulative EXP
1421	0
1422	515381578
1497	515381578
1498	1226796007
1716	1226796007
1717	1368999836
1795	1368999836

`encode()` output:

```
Leviathan[4111]1380?Deep Current@Sunken Arch:1422$Batch 0 Kelp:1434009068&Grade
```

- **Sub-case 2**

```
Nova Shell[4313]23?Fog Bank@Sunken Arch:24:39$Gems:584633497&Crystals:202209422%
```

Top-level: `shipName = "Nova Shell"` , `equipment = {4,3,1,3}` , `stops.size() =3`,
`departMinutes =1380`

Stops:

#	location	arriveMinutes	hazards	exp	leaveMinutes
0	"Sunken Arch"	1479	Fog Bank	456423512	1562
1	"Hex 0 Atoll"	1648	Whirlpool	415252392	1787
2	"Site 8 Hollow"	1831	Dark Current	317796632	1990

`getExpAt` (selected): 1478 → 0, 1479 → 456423512, 1648 → 871675904, 1831 → 1189472536

`encode()` output:

```
Nova Shell[4313]1380?Fog Bank@Sunken Arch:1479$Gems:584633497&Crystals:2022094229
```

- **Sub-case 3**

```
Nautilus[3332]23?Storm Surge@Storm Cape:24:29$Driftwood:545178500&Saltstone:1087
```

Top-level: `shipName = "Nautilus"` , `equipment = {3,3,3,2}` , `stops.size() =3`,
`departMinutes =1380`

Stops:

#	location	arriveMinutes	hazards	exp	leaveMinutes
0	"Storm Cape"	1469	Storm Surge	330222870	1560
1	"Block 2 Shelf"	1615	Ice Vortex	490974549	1778
2	"Ember Isle"	1809	Lava Flow	376500717	1962

`getExpAt` (selected): 1468 → 0, 1469 → 330222870, 1615 → 821197419, 1809 → 1197698136

`encode()` output:

```
Nautilus[3332]1380?Storm Surge@Storm Cape:1469$Driftwood:545178500&Saltstone:1087
```

Requirements

Implement the four methods declared in `VoyageLog.h` .

- `parse`
- `getLocationAt`
- `getExpAt`
- `encode`

Input

1. Complete the class implementation in `VoyageLog.h` .
2. All test logs are guaranteed to be syntactically valid. Item quantities and individual EXP values are guaranteed to fit within the range of `int` .
3. The Online Judge will substitute the following files:
 - `entrypoint.cpp`
 - `case.h`
 - `test.h`
4. The test cases for student available in `case*.h` .

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.